

## Communication protocols:

- **Communication Protocol** = Set of rules for data exchange.
- Types: **Serial (UART, I<sup>2</sup>C, SPI), Parallel, Network (TCP/IP), Wireless (Wi-Fi, Bluetooth, etc.)**.
- Real-world: Like a **language agreement** between devices.

## ◆ What is I<sup>2</sup>C?

- **I<sup>2</sup>C (Inter-Integrated Circuit)** is a communication protocol.
- It is used for **short-distance communication** between devices (ICs) on the same circuit board.
- It was invented by **Philips** in 1982.
- Only needs **2 wires** for communication:
- **SDA (Serial Data Line)** → carries the data.
- **SCL (Serial Clock Line)** → carries the clock signal.

## ◆ Who Uses It?

- **Microcontrollers** talk to **sensors, EEPROMs, displays, and other chips**.
- Example: In your **phone**, I<sup>2</sup>C connects the **processor** to **touchscreen, camera sensor, and battery controller**.

## ◆ How It Works

- **Master & Slave Concept**
- One device acts as the **Master** (usually microcontroller).
- Other devices are **Slaves** (sensors, displays, memory chips).
- Master controls the clock (SCL) and communication.

👉 Example: Imagine a **teacher (Master)** asking **students (Slaves)** questions one by one.

- **Unique Address**
- Each slave device has a **7-bit or 10-bit address**.

- Master uses this address to talk to the correct device.
- Example: In a classroom, each student has a **roll number**. Teacher calls by roll number.
- **Data Transfer**
- Master sends a **START** signal.
- Master sends the **slave address** + Read/Write bit.
- If the slave is present, it replies with **ACK (Acknowledge)**.
- Data is sent **bit by bit** on SDA, in sync with SCL clock.
- Transfer ends with a **STOP** signal.

## ◆ Real-Time Example

- Suppose you are using an **Arduino + Temperature Sensor (LM75) + LCD Display**:
- Arduino (Master) asks LM75 (Slave with address 0x48) → "Give me temperature".
- LM75 replies with the data.
- Arduino then sends the data to the LCD (Slave with address 0x27) → "Show 25°C".
- All this happens on the **same two wires** (SDA, SCL).

## ◆ Features of I<sup>2</sup>C

- Only **2 wires** needed → saves PCB space.
- Can connect **many devices** (each with unique address).
- Supports different speeds:
- Standard Mode: 100 kbps
- Fast Mode: 400 kbps
- Fast Mode Plus: 1 Mbps
- High-Speed Mode: 3.4 Mbps
- Communication distance: only **a few meters**, best for inside a device

Example program:

## ◆ I<sup>2</sup>C Data Frame Example

Suppose we want to **read temperature** from a sensor (LM75) at **address 0x48**.

### Steps:

- Master sends **START**
- Master sends **Slave Address (0x48) + Write (0)** → 1001000 0
- Slave sends **ACK**
- Master sends **Register Address (0x00)** → temperature register
- Slave sends **ACK**
- Master sends **STOP**

👉 Then for reading:

- Master sends **START**
- Master sends **Slave Address (0x48) + Read (1)** → 1001000 1
- Slave sends **ACK**
- Slave sends **Data Byte (e.g., 0x19 = 25°C)**
- Master sends **ACK**
- Master sends **STOP**
- 

### Frame (Write then Read):

START → 10010000 (0x90) → ACK → 00000000 (reg addr) → ACK → STOP

START → 10010001 (0x91) → ACK → 00011001 (temp=25) → NACK → STOP

## Embedded C Example Program (Bit-banging I<sup>2</sup>C)

```
#include <reg51.h>
```

```
// SDA and SCL pins
```

```
sbit SDA = P2^0;
```

```
sbit SCL = P2^1;
```

```
void I2C_Delay() {  
    unsigned int i;  
    for(i=0;i<100;i++); // Small delay  
}
```

```
void I2C_Start() {  
    SDA = 1; SCL = 1; I2C_Delay();  
    SDA = 0; I2C_Delay();  
    SCL = 0;  
}
```

```
void I2C_Stop() {  
    SDA = 0; SCL = 1; I2C_Delay();  
    SDA = 1; I2C_Delay();  
}
```

```
void I2C_Write(unsigned char dat) {  
    unsigned char i;  
    for(i=0;i<8;i++) {  
        SDA = (dat & 0x80) >> 7; // Send MSB first  
        SCL = 1; I2C_Delay();  
        SCL = 0; I2C_Delay();  
        dat <<= 1;  
    }  
    SDA = 1; SCL = 1; I2C_Delay(); // ACK cycle  
    SCL = 0;
```

```
}
```

```
unsigned char I2C_Read() {  
    unsigned char i, dat=0;  
    SDA = 1; // Release SDA for input  
    for(i=0;i<8;i++) {  
        SCL = 1; I2C_Delay();  
        dat = (dat<<1) | SDA; // Read bit  
        SCL = 0; I2C_Delay();  
    }  
    return dat;  
}
```

```
void main() {  
    unsigned char temp;  
  
    // Write cycle: select register 0x00  
    I2C_Start();  
    I2C_Write(0x90); // Slave address (0x48<<1 | Write=0)  
    I2C_Write(0x00); // Temperature register  
    I2C_Stop();  
  
    // Read cycle  
    I2C_Start();  
    I2C_Write(0x91); // Slave address (0x48<<1 | Read=1)  
    temp = I2C_Read(); // Read data byte  
    I2C_Stop();  
  
    while(1); // Debug: check temp in debugger/watch  
}
```



## How Code Matches Data Frame

- I2C\_Start() → **START** condition
- I2C\_Write(0x90) → Address + Write frame
- I2C\_Write(0x00) → Register address frame
- I2C\_Stop() → **STOP**
- I2C\_Write(0x91) → Address + Read frame
- temp = I2C\_Read() → Data byte frame

Thank you.







